

Atelier TP — Soirée 17

Structures de données objet — tris, recherches et collections sur un catalogue de **Livre**

Cet atelier se déroule dans un **dépôt GitHub**. **Forkez** le dépôt du cours, **clonez votre fork** dans VS Code, puis renommez le squelette `catalogue_squelette.py` en `catalogue.py` et complétez-y les fonctions au fil des exercices. La procédure Git complète (fork, commits, push, remise) figure dans le `README.md` du dépôt.

La classe `Livre` (`livre_s17.py`) est fournie dans le dépôt : ne la modifiez pas. Toutes les fonctions reçoivent une **liste de Livre** et n'ont pas le droit de la modifier : produisez toujours une nouvelle collection. Aucune classe n'est à écrire ce soir.

Réponses ouvertes. Les exercices 1, 3 et 5 demandent une réponse *écrite* (pas seulement du code). Rédigez-les dans le fichier `reponses.txt` fourni dans le dépôt.

Construisez ce catalogue en tête de vos tests ; tous les exemples d'énoncé s'y rapportent.

```
from livre_s17 import Livre

CATALOGUE = [
    Livre("1984", "Orwell", "9780451524935", 328, 1949),
    Livre("La Ferme des animaux", "Orwell", "9780141036137", 112, 1945),
    Livre("Le Meilleur des mondes", "Huxley", "9780060850524", 311, 1932),
    Livre("Fahrenheit 451", "Bradbury", "9781451673319", 256, 1953),
]
```

Exercice 1 — Diagnostic d'ouverture (AA3)

Aucun code à écrire dans cet exercice. Le diagnostic précède la correction.

1. Exécutez `tri_buggy.py`. Le programme affiche un compte surprenant : un livre a disparu.
2. Recopiez les deux comptes affichés (livres ajoutés / livres distincts) et la liste des titres conservés.
3. Répondez par écrit dans `reponses.txt` (voir « Réponses ouvertes » de la mise en place), en deux ou trois phrases :
 - Combien de livres ont été ajoutés ? Combien l'ensemble (`set`) en garde-t-il ? Lequel a disparu ?
 - Pourquoi un `set` considère-t-il deux livres aux **titres différents** comme un seul et même livre ? Qu'est-ce qui, pour un ensemble, rend deux objets « identiques » ?
 - Quelles méthodes de `Livre` sont en jeu ? S'agit-il d'un bug ou d'une conséquence d'une décision de conception ? Justifiez en une phrase.

Les deux comptes (par ex. 5 ajoutés, 4 distincts) et les titres sont recopiés sans paraphrase. Le diagnostic identifie que le `set` s'appuie sur `__eq__` et `__hash__` de `Livre` (fondés sur l'ISBN), que le titre n'entre pas dans cette comparaison, et relie ce comportement à la décision d'identité prise en soirée 12 (ce n'est pas un bug).

Exercice 2 — Trier avec `sorted(key=...)`

Complétez les trois fonctions de tri. Chacune retourne une **nouvelle** liste ; `CATALOGUE` doit rester inchangé après l'appel.

1. `trier_par_titre(livres)` — trie par titre, ordre alphabétique croissant.
2. `trier_par_auteur_puis_titre(livres)` — trie par auteur croissant ; à auteur égal, par titre croissant.
3. `trier_par_annee(livres, recents_dabord=False)` — trie par année croissante. Si `recents_dabord` vaut `True`, les livres les plus récents apparaissent en premier.

Résultats attendus sur `CATALOGUE` :

```
[l.titre for l in trier_par_titre(CATALOGUE)]
# ['1984', 'Fahrenheit 451', 'La Ferme des animaux', 'Le Meilleur des mondes']

[l.annee for l in trier_par_annee(CATALOGUE)]
# [1932, 1945, 1949, 1953]

[l.annee for l in trier_par_annee(CATALOGUE, recents_dabord=True)]
# [1953, 1949, 1945, 1932]
```

Les trois fonctions utilisent `sorted` avec une fonction-clé. La clé composée de la fonction 2 est un `tuple` (auteur, titre) . La fonction 3 pilote l'argument `reverse` . `CATALOGUE` est intact après chaque appel (vérifiez-le).

Exercice 3 — Clé composée à directions opposées (AA1 + AA3)

Consigne. Complétez `trier_par_auteur_puis_annee_recente(livres)` : les auteurs en ordre croissant (A vers Z) ; à auteur égal, les livres du **plus récent au plus ancien**.

Résultat attendu sur `CATALOGUE` :

```
[(l.auteur, l.annee) for l in trier_par_auteur_puis_annee_recente(CATALOGUE)]
# [('Bradbury', 1953), ('Huxley', 1932), ('Orwell', 1949), ('Orwell', 1945)]
```

Justification écrite (AA3) — à rédiger dans `reponses.txt` . Expliquez en deux phrases pourquoi `reverse=True` ne convient **pas** ici, et comment la négation du critère « année » résout le problème.

La fonction trie correctement (Orwell : 1949 avant 1945). La justification distingue clairement l'effet de `reverse` (sur la clé entière) de celui de la négation (sur le seul critère numérique « année »).

Exercice 4 — Rechercher (AA1)

1. `rechercher_par_auteur(livres, auteur)` — retourne la **liste** de tous les livres dont l'auteur correspond exactement. Liste vide si aucun.
2. `rechercher_par_isbn(livres, isbn)` — retourne le **premier** livre dont l'ISBN correspond, ou `None` s'il est absent. La fonction s'arrête dès qu'elle l'a trouvé.

Résultats attendus :

```
len(rechercher_par_auteur(CATALOGUE, "Orwell"))      # 2
rechercher_par_auteur(CATALOGUE, "Inconnu")          # []
rechercher_par_isbn(CATALOGUE, "9780451524935").titre # '1984'
rechercher_par_isbn(CATALOGUE, "0000000000000")      # None
```

La recherche par auteur retourne une liste (éventuellement vide), jamais `None`. La recherche par ISBN retourne un `Livre` ou `None`, et sort de la boucle au premier livre trouvé.

Exercice 5 — Dédoublonner avec un ensemble (AA1 + AA3)

Ajoutez à votre jeu de tests un **doublon métier** (même ISBN que « 1984 », titre saisi différemment) :

```
DOUBLON = Livre("1984 (réédition)", "Orwell", "9780451524935", 328, 1949)
AVEC_DOUBLON = CATALOGUE + [DOUBLON]
```

1. `compter_distincts(livres)` — retourne le nombre de livres **distincts**.
2. `dedoublonner(livres)` — retourne une liste sans doublon, en conservant l'ordre de **première apparition**.

Résultats attendus :

```
compter_distincts(AVEC_DOUBLON)      # 4
len(dedoublonner(AVEC_DOUBLON))      # 4
dedoublonner(AVEC_DOUBLON)[0].titre  # '1984' (pas la réédition)
```

Justification écrite (AA3) — à rédiger dans `reponses.txt`. Pourquoi deux `Livre` de même ISBN sont-ils dédoublonnés, alors que leurs titres diffèrent ? Quelles méthodes de `Livre` rendent cela possible ? Que se passerait-il si `Livre` ne les définissait pas ?

`compter_distincts` s'appuie sur `set`. `dedoubler` conserve l'ordre (indice : `dict.fromkeys`). La justification cite `__eq__` et `__hash__` (par ISBN) et indique que, sans eux, le dédoublement échouerait (comparaison par identité mémoire).

Exercice 6 — Indexer et regrouper (AA1)

1. `indexer_par_isbn(livres)` — retourne un dictionnaire `{isbn: livre}` permettant un accès direct par ISBN.
2. `regrouper_par_auteur(livres)` — retourne un dictionnaire `{auteur: [livres]}`. À l'intérieur d'un groupe, les livres conservent leur ordre d'apparition. Utilisez `setdefault`.

Résultats attendus :

```
indexer_par_isbn(CATALOGUE)["9780060850524"].titre # 'Le Meilleur des mondes'

g = regroupier_par_auteur(CATALOGUE)
len(g["Orwell"]) # 2
[1.titre for l in g["Orwell"]] # ['1984', 'La Ferme des animaux']
```

Pour aller plus loin (facultatif). Réécrivez la fonction `regrouper_par_auteur` avec un `defaultdict(list)` (du module `collections`) et vérifiez que le résultat est identique.

L'index donne un accès direct `index[isbn]`. Le regroupement utilise `setdefault` et préserve l'ordre dans chaque groupe. La variante `defaultdict` (si traitée) produit un résultat égal au premier.

Exercice 7 — Tester ses fonctions (AA2)

Écrivez, dans un fichier `test_catalogue.py`, une suite `unittest` couvrant vos fonctions. Au minimum :

- un test d'ordre pour chaque fonction de tri (comparez la liste des titres ou des années obtenue à la liste attendue) ;
- un test de **non-modification** : `CATALOGUE` est inchangé après un tri ;
- un test de **stabilité** : deux livres de même année conservent leur ordre d'entrée ;
- un test de dédoublement (taille et ordre) ;
- un test de regroupement (clés présentes et taille des groupes).

La suite s'exécute par `python -m unittest test_catalogue` et tous les tests passent. Chaque test vérifie un comportement **observable** (ordre, taille, contenu), pas l'implémentation interne.

La remise se fait **par pull request** vers le dépôt du cours (procédure dans le `README.md`).
Versionnez et poussez vers votre fork : `catalogue.py` (complété), `test_catalogue.py` (vos tests) et `reponses.txt` (vos réponses écrites). Un commit par exercice est conseillé ; la cadence figure dans le `README.md` .